Ders 4 Genişletilmiş Notu: JK ve T Flip-Flop ile Sayıcı Tasarımı, Kendi Kendini Düzeltme, ROM ve PLA

Ders 4 Genişletilmiş Notu: JK ve T Flip-Flop ile Sayıcı Tasarımı, Kendi Kendini Düzeltme, ROM ve PLA

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Dersin Planı
- 2. JK Flip-Flop ile Sayıcı Tasarımı
- 3. T Flip-Flop ile 0, 1, 2, 4, 7 Sayıcısı
- 4. Kullanılmayan Durumlar ve Kendi Kendini Düzeltme
- 5. Programlanabilir Lojik Cihazlara Giriş
- 6. ROM (Read-Only Memory)
- 7. Küçük Bir Örnek: 8×4 ROM
- 8. ROM ile Kombinasyonel Devre: 3 Bitlik Sayının Karesi
- 9. ROM Boyutu Soruları
- 10. PLA (Programmable Logic Array)
- 11. PLA Tasarımında Ortak Terim Aramak

1. Dersin Planı

Dönemin dördüncü haftasında hoca dersi iki parçaya ayırdı. İlk bölümde geçen haftadan kalan, farklı sayma düzenlerinde sayan sayıcı örnekleri işlendi. İkinci bölümde programlanabilir lojik cihazlar (programmable logic devices) konusuna geçildi.

2. JK Flip-Flop ile Sayıcı Tasarımı

2.1 JK ile Tasarımda Neden Ek Sütun Gerekir?

İlk örnek, geçen hafta başlanan JK flip-flop'lu sayıcıydı. Hoca bu örnekte özellikle geçiş tablosunun nasıl hazırlandığını vurgulamak istediğini söyledi.

Soru: Geçiş tablosunu hazırladıktan sonra durum diyagramı (state diagram) mı çizeceğiz? **Cevap:** Hayır. Tabloyu tamamladıktan sonra Karnaugh haritasına taşıyacaksınız ve her JK girişinin indirgenmiş fonksiyonunu bulacaksınız. Çözüm bu şekilde olacak.

Soru: Burada bir sayma yapıyoruz. "Sayı" yalnızca hangi durumda olduğumuzu mu gösteriyor, yoksa bir yerde ayrıca bir toplama da yapıyor muyuz? **Cevap:** Ekrandaki tablo bir durum tablosudur (state table). İçinde mevcut durum (present state) ve sonraki durum (next state) vardır. Şimdiye kadar hep D tipi flip-flop'la tasarım yaptığımız için, sonraki durum sütunu zaten D flip-flop'un girişini gösteriyordu. Bu yüzden ayrıca bir flip-flop girişi yazmamız gerekmiyordu. JK flip-flop ile tasarımda iş değişir: Flip-flop girişlerini tabloya ayrıca yazmamız gerekir.

2.2 Geçiş Tablosunu Fonksiyon Tablosundan Çıkarmak

> **Hocanın vurgusu:** Geçiş tablosunu ezberlemeniz gerekmiyor, kendiniz çıkaracaksınız. Yalnızca JK flip-flop'un fonksiyon tablosunu biliyor olmanız yeterli. Öbürünü

oradan çıkarabilirsiniz.

Geçiş tablosuna neden ihtiyaç var? Çünkü durum tablosunda bir flip-flop'un mevcut durumunu ve sonraki durumunu biliyoruz. Örneğin A_2 bitinin mevcut durumu 0, sonraki durumu da 0 olsun. Bu durumda A_2 bitini oluşturan flip-flop'un J ve K girişlerine ne yazılacağını bilmemiz gerekir. Fonksiyon tablosu bunun tersini söyler: Girişler verilince çıkışın ne olacağını gösterir. Bu yüzden her satır için şu soruyu sorarız: "Mevcut durum $Q(t)$ iken sonraki durumun $Q(t+1)$ olması için JK ne olmalıdır?"

Hocanın kullandığı fonksiyon tablosunun dört satırı şudur (ara adım eklendi; hoca satırları ekranda göstererek okudu):

J	K	$Q(t+1)$	Anlamı
0	0	$Q(t)$	Durum korunur
0	1	0	Sıfırlanır
1	0	1	Birlenir
1	1	$Q'(t)$	Tümlenir

Geçiş tablosu bu satırlardan şöyle çıkar:

- $0 \to 0$**:** Sonraki durumun 0 olması ya da sonraki durumun mevcut durumla aynı olması gerekir. Bunu iki satır sağlar ($JK = 01$ ve $JK = 00$). İkisinde de $J = 0$, K ise fark etmez. Sonuç: $0X$.
- $0 \to 1$**:** Sonraki durumun 1 olması ya da mevcut durumun tümleyeni olması gerekir ($JK = 10$ ve $JK = 11$). İkisinde de $J = 1$. Sonuç: $1X$.
- $1 \to 0$**:** Sonraki durumun 0 olması ya da mevcut durumun tümleyeni olması gerekir ($JK = 01$ ve $JK = 11$). İkisinde de $K = 1$. Sonuç: $X1$.
- $1 \to 1$**:** Kalan kombinasyon aynı yolla çıkar. Sonuç: $X0$.

2.3 A_2 Biti İçin J_2 ve K_2 Sütunları

Geçiş tablosu hazır olduğuna göre flip-flop girişlerini yazabiliriz. Sayıcının her biti için ayrı bir JK çifti vardır. A_2 bitine bakıp J_2 ve K_2 sütunlarını satır satır doldururuz. Hoca bu sütunu öğrencilerle birlikte tamamladı.

Mevcut durum $A_2A_1A_0$	A_2 geçişi	J_2	K_2
000	$0 \to 0$	0	X
001	$0 \to 0$	0	X
010	$0 \to 1$	1	X
011	$0 \to 0$	0	X
100	$1 \to 1$	X	0
101	$1 \to X$	X	X
110	$1 \to 0$	X	1
111	$1 \to X$	X	X

Mevcut durumu 1, sonraki durumu "fark etmez" olan satırlarda hem J hem K fark etmez olur, yani XX yazılır.

2.4 J_2 ve K_2'nin İndirgenmesi

Her flip-flop girişi için ayrı bir doğruluk tablosu, dolayısıyla ayrı bir Karnaugh haritası gerekir. Haritada satır A_2, sütunlar A_1A_0 ile etiketlenir. Sütun etiketleri $00, 01, 11, 10$ sırasındadır. Bu yüzden gözlerin değerleri soldan sağa $0, 1, 3, 2$ diye gider. Tablodaki satırlar haritaya bu sırayla taşınır.

J_2 haritası:

$A_2 \backslash A_1A_0$	00	01	11	10
0	0	0	0	1
1	X	X	X	X

Sağ sütundaki iki göz gruplanır. Bu grupta $A_1 = 1$, $A_0 = 0$ sabit kalır:

$$J_2 = A_1 A_0'$$

K_2 için hoca önce kestirme bir yol olup olmadığına baktı: Birinci satır tümüyle fark etmez.

$A_2 \backslash A_1A_0$	00	01	11	10
0	X	X	X	X
1	0	X	X	1

Sağdaki iki sütun dörtlü bir grup oluşturur. Bu dörtlüde değişmeyen bit A_1'dir:

$$K_2 = A_1$$

Ders sonrası not: Hoca önce $K_2 = A_0$ yazdı. Haritanın son sütununu yanlışlıkla 01 diye etiketlemişti. Bir öğrenci etiketin 10 olması gerektiğini gösterdi, hoca da düzeltti. Doğrusu $K_2 = A_1$. Python kontrolü: $K_2 = A_0$ seçimi 110 durumunda 0 verir, oysa orada $K_2 = 1$ gerekir.

Hocanın vurgusu: Dikkatli olmakta fayda var. Küçücük bir hata, sonucu yanlış yazmanıza sebep olabilir.

Bu işlem bir flip-flop içindir. Hoca, diğer iki flip-flop için aynı işlemlerin benzer şekilde yapılabileceğini söyleyip örneği burada bıraktı.

3. T Flip-Flop ile 0, 1, 2, 4, 7 Sayıcısı

3.1 Soru ve T Flip-Flop'un Davranışı

İkinci örnekte sayıcının $0, 1, 2, 4, 7$ düzeninde saymasını ve bu düzeni tekrar etmesini (repeated sequence) istiyoruz. Yani 7'den sonra tekrar 0'a döner. Hoca bu örneği, öncekinden farklı olarak, sonuna kadar çözdü. Çünkü sonunda bir özelliği, kendi kendini düzeltme özelliğini arayacağız.

T flip-flop'un fonksiyonu basittir:

- $T = 0$ iken sonraki durum mevcut durumla aynıdır.
- $T = 1$ iken sonraki durum mevcut durumun tümleyenidir.

Bu yüzden ayrı bir geçiş tablosu yazmaya gerek yoktur; kavramdan gidilebilir. Bit değişmiyorsa $T = 0$, değişiyorsa $T = 1$ yazarız. Zorlananlar geçiş tablosunu yine de kullanabilir.

3.2 Flip-Flop Giriş Tablosu

Kullanılmayan durumlarda (unused states) sonraki durum belirtilmediği için T girişleri fark etmez olur. Hoca sütunları öğrencilerle birlikte doldurdu:

Mevcut $A_2A_1A_0$	Sonraki $A_2A_1A_0$	T_2	T_1	T_0
000	001	0	0	1
001	010	0	1	1
010	100	1	1	0
011	-	X	X	X
100	111	0	1	1

Mevcut $A_2A_1A_0$	Sonraki $A_2A_1A_0$	T_2	T_1	T_0
101	-	X	X	X
110	-	X	X	X
111	000	1	1	1

Hoca, üç sütunun hiçbirinin tümüyle 0 ya da 1 olmadığına bakıp üçü için de Karnaugh haritası gerektiğini söyledi.

3.3 Karnaugh Haritaları

T_2:

$A_2 \backslash A_1A_0$	00	01	11	10
0	0	0	X	1
1	0	X	1	X

Sağdaki iki sütunun oluşturduğu dörtlü grup:

$$T_2 = A_1$$

T_1:

$A_2 \backslash A_1A_0$	00	01	11	10
0	0	1	X	1
1	1	X	1	X

Haritada tek bir 0 vardır. Hoca bunu iki yoldan okudu. Tek sıfırı gruplarsak $T_1' = A_2'A_1'A_0'$ çıkar, bunun tümleyeni de aranan ifadedir. Ya da üç dörtlü grup alırız: A_1 dörtlüsü, A_2 dörtlüsü ve A_0 dörtlüsü. İki yol da aynı sonucu verir:

$$T_1 = A_2 + A_1 + A_0$$

Soru: A_0 diyebilmemiz için hem en sağdaki ikiliyi hem en soldaki ikiliyi alabiliyor olmamız gerekmez mi? **Cevap:** Hoca gruplara tek tek baktı ve A_0 grubunun alınabildiğini gösterdi.

> **Ders sonrası not:** Sütun etiketlerine göre $A_0 = 1$ olan dörtlü, orta iki sütundan (01 ve 11) oluşur. Uçtaki iki sütun (00 ve 10) ise A_0' grubudur. Bu nedenle A_0 grubu için uç sütunlara gerek yoktur.

T_0:

$A_2 \backslash A_1A_0$	00	01	11	10
0	1	1	X	0
1	1	X	1	X

İki dörtlü grup yazılır: İlk iki sütun A_1', alttaki satır A_2 verir:

$$T_0 = A_1' + A_2$$

3.4 Öğrencinin XOR Önerisi

Soru: Don't care'leri 0 yazınca T_1 doğrudan $A_2 \oplus A_1 \oplus A_0$ çıkmıyor mu? **Cevap:** Hoca önce XOR çıkmadığını söyledi. Öğrenci sorusunu açınca, bütün fark etmezleri 0 aldığında XOR'un doğru olduğunu kabul etti. Kendisi T_1 için bütün fark etmezleri 1 almıştı. Aradaki fark buradan geliyordu. İkisi de doğru birer alternatiftir.

> **Ders sonrası not:** Hoca ilk cevabında "XOR çıkmıyor" dedi; öğrencinin önerisi doğrudur. Fark etmezler 0 alınınca T_1 yalnız 1, 2, 4 ve 7 durumlarında 1 olur. Bu durumların ikili yazımında tek sayıda 1 vardır, bu yüzden $T_1 = A_2 \oplus A_1 \oplus A_0$ olur. Python ile sekiz satırda doğrulandı.

Hoca bu noktayı bir sonraki konuya bağladı: Fark etmez koşullarını 0 ya da 1 kabul etmenin artıları ve eksileri vardır.

4. Kullanılmayan Durumlar ve Kendi Kendini Düzeltme

4.1 Sorun: Devre Hangi Durumdan Başlar?

Tasarladığımız sayıcı $0 \to 1 \to 2 \to 4 \to 7 \to 0$ düzeninde sayıyor. Bu düzende 3, 5 ve 6 hiç geçmez; bunlar kullanılmayan durumlardır.

Devreye ilk güç uygulandığında sistem resetlenmeden herhangi bir noktadan çalışmaya başlarsa, bu kullanılmayan durumlardan biriyle başlayabilir. O zaman sonraki durumun ne olacağı, fark etmezleri 0 mı 1 mi kabul ettiğimize bağlıdır.

Amacımız şudur: Devre kullanılmayan bir durumdan başlasa bile kullanılan durumlardan birine geçmelidir. Kullanılmayan durumlar kendi aralarında bir döngü oluşturursa, örneğin $3 \to 5$, $5 \to 6$, $6 \to 3$ olursa, devre bu döngüye girer, kullanılan durumlara geçemez ve kilitlenir kalır.

> **Ders sonrası not:** Altyazıda hocanın "sistem unused state'lere geçebilsin" dediği yer var. Bir önceki cümleden anlaşıldığı gibi amaç **used** durumlara geçmektir.

Hoca, tasarımı bu konuyu hiç dikkate almadan yaptığını belirtti. Şimdi kendi kendini düzeltip düzeltmediğine bakılacak.

4.2 Kontrol

Her kullanılmayan durum için haritaya döneriz ve o gözde fark etmezin 0 mı 1 mi alındığına bakarız. Gruba girmiş göz 1, girmemiş göz 0 alınmıştır. Sonra $T = 1$ olan bitleri tümleyerek sonraki durumu buluruz.

Durum 3 (011): $T_2 = 1$, $T_1 = 1$, $T_0 = 0$ alınmış. A_2 biti $0 \to 1$, A_1 biti $1 \to 0$ olur, A_0 biti değişmez ve 1 kalır. Sonraki durum 101, yani 5. Kullanılmayan bir durumdan yine kullanılmayan bir duruma geçtik. Bu yüzden 5'e de bakmamız gerekir.

Durum 5 (101): $T_2T_1T_0 = 011$. $A_2 = 1$ kalır, A_1 biti $0 \to 1$, A_0 biti $1 \to 0$ olur. Sonraki durum 110, yani 6. Bu da kullanılmayan bir durumdur. Şimdi her şey 6'ya bağlı: 6 kullanılan bir duruma geçerse sorun yoktur, kullanılmayan bir duruma geçerse kapalı bir çevrim oluşur.

Durum 6 (110): $T_2T_1T_0 = 111$. Bütün bitlerin tümleyeni alınır. Sonraki durum 001, yani 1. Bu kullanılan bir durumdur.

Sonuç olarak zincir şudur:

$$3 \to 5 \to 6 \to 1$$

Kullanılmayan durum	$T_2T_1T_0$	Sonraki durum
011 (3)	110	101 (5)
101 (5)	011	110 (6)
110 (6)	111	001 (1)

Devre kendi kendini düzeltme özelliğine (self-correcting) sahiptir. Python benzetimi de bu geçişleri doğruladı.

> **Ders sonrası not:** Öğrencinin XOR çözümünde T_1 değiştiği için geçişler de değişir. $T_2 = A_1$ ve $T_0 = A_1' + A_2$ aynı kalırsa python benzetimi $3 \to 7$, $5 \to 4$, $6 \to 3$ verir. 6 önce 3'e, oradan 7'ye geçer. Bu seçim de kendi kendini düzeltir. Bu kontrol derste yapılmadı. Fark etmez seçiminin geçişleri nasıl değiştirdiğini burada görebilirsiniz.

4.3 Sağlamasaydı Ne Yapardık?

Hoca sordu: Kontrol olumsuz çıksaydı ne yapardık? Bir öğrencinin cevabı doğruydu: Fark etmezleri değiştirip yeniden tasarlardık. Hoca bunu şöyle açtı: Örneğin 6, 3'e gidiyor olsaydı, bunu önlemek için T girişlerinde 0 ya da 1 kabul ettiğimiz fark etmezleri değiştirirdik. Bu, belki daha karmaşık bir devre oluşmasını göze almak demektir. Tasarımda bu tür istekler olabilir ve dikkate alınmaları gerekir.

4.4 Devreye Geçiş

İki örnekle hem JK flip-flop'la hem T flip-flop'la tasarım görülmüş oldu. Bu sayıcıyı gerçeklemek için 3 tane T tipi flip-flop kullanılır ve her flip-flop'un girişi bulunan fonksiyonlara göre bağlanır.

5. Programlanabilir Lojik Cihazlara Giriş

Hoca dört programlanabilir lojik cihaz sıraladı: ROM (Read-Only Memory), PLA (Programmable Logic Array), PAL (Programmable Array Logic) ve FPGA gibi yapıların bulunduğu CPLD (Complex Programmable Logic Device). Ders kapsamında ilk üçü anlatılacak.

Bu yapıların ortak noktası şudur: Hepsinde bir AND dizisi (AND array) ve bir OR dizisi (OR array) vardır. Hangi dizinin önündeki yapının programlanabilir, hangisinin sabit olduğu, cihazın türünü belirler.

Yapı	AND dizisinin önü	OR dizisinin önü
ROM (PROM)	Sabit (dizi decoder ile oluşturulur)	Programlanabilir
PAL	Programlanabilir	Sabit
PLA	Programlanabilir	Programlanabilir

PLA, iki dizinin önü de programlanabildiği için en esnek yapıdır. Anlatım sırası ROM, PLA, PAL olacak.

6. ROM (Read-Only Memory)

6.1 Defter Sayfası Benzetmesi

Hoca ROM'u anlatırken öğrencilerine hep aynı şeyi söylediğini belirtti: ROM'u bir defterin sayfası gibi düşünün. Bu sayfada hangi satıra ne yazdığınızı bilmek istersiniz. Bunun için bir satır numarasını tutan adres bilgisine ve o satırda ne yazdığını gösteren bilgiye ihtiyacımız vardır. ROM, bilgisayarın içinde de kullandığımız bir bellek yapısıdır.

6.2 Blok Yapısı ve İç Yapı

ROM blok olarak $2^k \times n$ biçiminde gösterilir:

- k: Adres uçlarının sayısıdır. Bu uçlar sisteme giriş olarak bağlıdır ve satırı seçer.
- n: Seçilen adreste kayıtlı değerin bit sayısıdır, yani veri çıkışıdır.

İç yapıda sabit AND dizisi decoder'ın içindedir. Programlanabilir OR dizisi ise decoder çıkışlarıyla OR kapıları arasındaki yapıdır.

Hocanın üzerinde durduğu şekil 5 adres girişli ve 8 veri çıkışlıdır. 5 bitlik adres, 5'ten 32'ye bir decoder ile 32 hatta genişletilir. Örneğin adres 5 ise yalnız 5. satır 1 olur, diğerleri 0 olur.

Decoder'ın 32 çıkışının hepsi her bir OR kapısına bağlanır. Yani her OR kapısı aslında 32 girişlidir.

Bunları ayrı ayrı çizmek devreyi çok karmaşık hale getireceği için dizi lojik gösterimi (array logic representation) kullanılır. OR kapısının tek bir giriş çizgisi yukarı doğru çizilir. Bu çizgiyi kesen 32 çizgi, kapının 32 girişi anlamına gelir.

6.3 Programlama: Sigortalar

Normalde ROM entegresi fabrika ortamında programlanır, ama değişik çeşitleri vardır: PROM, EPROM, EEPROM. Devre üzerinde de programlanabilen yapılar vardır.

> **Ek bilgi (derste anlatılmadı):** Bu adlardaki harfler büyük olasılıkla şu açılımlara karşılık gelir: PROM "programlanabilir", EPROM "silinebilir programlanabilir", EEPROM "elektrikle silinebilir programlanabilir" ROM. Silme yöntemi büyük olasılıkla EPROM'da morötesi ışık, EEPROM'da elektrik sinyalidir.

Programlanabilir bağlantılarda sigortalar (fuse) bulunur. Ürün elimize ilk geldiğinde ya da içindeki veri silindiğinde bütün sigortalar kapalı konumdadır. Yani decoder'ın bütün çıkışları OR kapılarına bağlıdır. Programlarken bağlı kalmasını istemediğimiz sigortaları patlatırız (blown). Hangi girişlerin OR kapısına bağlı olduğu, sigortaların bu konumlarına göre belirlenir.

Terimler karışmasın diye:

Gösterim	Terim	Sigorta	Bağlantı
×	Fuse intact	Kapalı	Bağlı
Çarpısız	Fuse blown	Açık	Bağlı değil

6.4 Bir Satırın Okunması

Hoca şekildeki mavi çarpılarla işaretli bir numaralı satırı inceledi. Adres girişine 00001 verildiğinde decoder'ın bir numaralı çıkışı aktif olur. Bu hatta lojik 1, bütün diğer hatlarda 0 görülür. Seçilen satırda kapalı olan sigortalardan OR kapısına 1 gider. Kapalı sigortaların yerine göre, en yüksek anlamlı A_7 bitinden başlayarak bu satırda kayıtlı değer $1001\,0011$ okunur.

Peki bu satırda sigortası açık olan bir OR kapısının çıkışı neden 0'dır? O kapının başka satırlarda kapalı sigortaları olabilir. Mesela A_6 bitini oluşturan OR kapısı için durum budur. Cevap decoder'dadır: Decoder'ın yalnız bir satırı lojik 1 olabilir, diğer bütün satırlarında 0 vardır. Sigorta kapalı olsa bile o satırdan OR kapısına 0 gelir ve çıkış 0 kalır. Seçili satırda kapalı olan sigortanın OR kapısında ise çıkış 1'dir.

Başka bir satır okumak için önce adres girişine o satırı vermek gerekir. Örneğin 30. satır için adres girişine 30 verilir. Sonra o satırdaki sigortaların konumuna göre kayıtlı değer bulunur.

Soru: Bunların analizini de yapacak mıyız, tablo oluşturup? **Cevap:** Evet. ROM'la ilgili örneklerde bu görülecek.

7. Küçük Bir Örnek: 8×4 ROM

8×4 ROM'u $2^3 \times 4$ ROM şeklinde yazmak daha çok kullanılan yoldur. Bu yazımdan 3 adres ucu olduğu ve her satırda 4 bit tutulduğu görülür. Yapıda 3'ten 8'e bir decoder (D_0 ... D_7 çıkışları) ve 4 OR kapısı vardır.

Örnekte ROM programlanmış olarak verilmiştir. Çarpı işareti fuse intact, çarpısız düz çizgi fuse blown demektir. Adres 001 için kayıtlı veri 0011 olarak okunur.

Hoca bu örnek üzerinden şu soruları sordu:

Soru: Bu OR kapıları kaç girişlidir? **Cevap:** 8 girişli. Decoder'ın 8 çıkışı var.

Soru: En yüksek anlamlı biti oluşturan F_3 için bu 8 girişten kaçı bağlıdır? **Cevap:** 3 tanesi.

Soru: Adres 1 seçilince D_1 çıkışı 1 oluyor. O halde F_3 ve F_2 neden 0 gösteriyor? **Cevap:** O satırda F_3 ve F_2 kapılarının sigortaları açıktır, kapılar bu satıra bağlı değildir.

Soru: Aynı OR kapısının kapalı olan diğer girişlerinden neden 1 gelmiyor? **Cevap:** O girişlerden işaret gelir, ama lojik 0 gelir. Çünkü yalnız D_1 çıkışı 1'dir. F_3 kapısının bağlı üç girişi bu satırlarda 0 olduğundan çıkış yine 0'dır.

8. ROM ile Kombinasyonel Devre: 3 Bitlik Sayının Karesi

Zaman zaman çok karmaşık devreleri ROM kullanarak tasarlayabiliriz. Bu örnekteki devre 3 bitlik bir sayı alır ve girişin karesine eşit bir ikili sayı üretir.

8.1 Doğruluk Tablosu

Girişler $A_2A_1A_0$'dır. En büyük giriş 7'dir ve $7 \times 7 = 49$ olur.

> **Ders sonrası not:** Hoca önce 49'un 5 bitle ifade edilebileceğini düşünmüştü, sonra düzeltti. Doğrusu 6 bittir ($49 = 110001_2$). Çıkışlar $S_5 \ldots S_0$ olarak yazılır.

Tablo öğrencilerle birlikte dolduruldu. Hoca değerleri ikilik ağırlıklarla kurdu: $25 = 16+8+1$, $36 = 32+4$, $49 = 32+16+1$.

Giriş $A_2A_1A_0$	Değer	Karesi	$S_5S_4S_3S_2S_1S_0$
000	0	0	000000
001	1	1	000001
010	2	4	000100
011	3	9	001001
100	4	16	010000
101	5	25	011001
110	6	36	100100
111	7	49	110001

Soru: Burada işlem yapan aritmetik bir devre kurmuyoruz değil mi? Sonuçları kendimiz hesaplayıp bir yerden okuyoruz. **Cevap:** Aynen öyle. Bir tabloya yazmış gibi oluyoruz ve devre çıkışı oradan okuyor. Arkada aritmetik işlem yapan bir devre yok. ROM tablosu ve içine yazdığımız değerler üzerinden sonucu üretiyoruz.

8.2 Optimizasyon: Daha Az Malzeme

Kullanılacak ROM'un 3 adres girişi olacaktır. Her satırda 6 bit tutmak gerekir gibi görünür, ama tablo incelenince iki bit için ROM'a gerek olmadığı görülür:

- $S_0 = A_0$: Sekiz satırda da aynıdır. Yeniden üretmeye gerek yoktur.
- $S_1 = 0$: Sütun tümüyle sıfırdır. Lojik 0'a bağlanır.

Tabloya yalnız yüksek anlamlı dört bit, $S_5S_4S_3S_2$ taşınır. Böylece kullanılacak ROM $2^3 \times 4$ olur.

8.3 ROM'un Programlanması

Yapıda adres girişleri A_2, A_1, A_0, 3'ten 8'e bir decoder, $D_0 \ldots D_7$ çıkışları ve 4 OR kapısı vardır. OR kapılarının çıkışları S_5, S_4, S_3, S_2'dir. Her satırda 1 olan bitin sigortası kapalı (×) bırakılır, 0 olan bitin sigortası patlatılır.

Satır	$S_5S_4S_3S_2$	S_5	S_4	S_3	S_2
D_0	0000				
D_1	0000				
D_2	0001				×
D_3	0010			×	
D_4	0100		×		
D_5	0110		×	×	
D_6	1001	×			×
D_7	1100	×	×		

Satır	$S_5S_4S_3S_2$	S_5	S_4	S_3	S_2

> **Ders sonrası not:** Altyazıda D_2 satırı için "A2 kapalı olacak" geçiyor; tabloya göre kapalı olan S_2 sigortasıdır. Hoca 6. satırı önce 1010 diye söyledi, sonra 1001 olarak düzeltti. Doğrusu 1001'dir, çünkü $36 = 100100_2$. Tablo python ile doğrulandı.

Bu şekilde ROM kullanarak bir kombinasyonel devre oluşturulmuş olur.

9. ROM Boyutu Soruları

Hoca kitabın bölüm sonu sorularından, belirli bir devreyi ROM ile gerçeklemek için gereken boyutu soran örnekler verdi. Yöntem hep aynıdır: Devrenin giriş sayısı k'yi, çıkış sayısı n'yi verir ve ROM boyutu $2^k \times n$ olur.

9.1 8 Bitlik Toplayıcı/Çıkarıcı

Devre iki 8 bitlik sayıyı hem toplayacak hem çıkaracak. Bunu seçen bir kontrol ucu ile elde girişi (carry in) ve elde çıkışı (carry out) uçları vardır. Hiç aritmetik devre kullanılmayacak.

Girişler: $8 + 8 = 16$ bit sayı girişi, 1 kontrol ucu ile 17 ve 1 carry in ile 18 bit.

Çıkışlar: 8 bit sonuç ve 1 bit carry out, toplam 9 bit.

$$\text{ROM boyutu} = 2^{18} \times 9$$

Soru: Bana 2^5 gibi geliyor, yani 32 girişli. **Cevap:** Sayıları da koymanız gerekir. 8 bitlik iki sayı girişi var, onları kısaltamazsınız. 18 girişiniz var.

Soru: Bir de carry out yok mu? O zaman 10 bit olmuyor mu? **Cevap:** Hayır. Sonuç 8 bittir, carry out ile birlikte 9 bit yazdık.

Soru: Neden 2^{18} olduğunu anlamadım. **Cevap:** ROM ile devre kurarken, kare alan devrede olduğu gibi bir tablo oluştururuz. Solda giriş, karşısında sonuç yazılır. Burada A sayısının 256 kombinasyonu, B sayısının da 256 kombinasyonu tabloya girer. Kontrol ve carry in bitleriyle 18 bitlik giriş ve 2^{18} farklı kombinasyon olur. Bu tablo elle yazılamayacak kadar büyüktür ($2^{18} = 262144$ satır; ara adım eklendi). Toplama devresi kullanmadan, lookup table gibi bir tablo üzerinden sonucu veren bir devre elde etmiş oluruz.

9.2 8 Bitlik Çarpıcı

İki 8 bitlik sayıyı çarpan devrede kontrol ucu yoktur. Giriş $8 + 8 = 16$ bittir. İki 8 bitlik sayının çarpımı 16 bitle ifade edilir.

$$\text{ROM boyutu} = 2^{16} \times 16$$

Soru: 15 bit yeterli olmaz mı? **Cevap:** Hoca kontrol edilmesini istedi ve kendi notunda 16 yazdığını söyledi.

> **Ders sonrası not:** Derste açıkta kalan bu sorunun cevabı: 15 bit yetmez. En büyük çarpım $255 \times 255 = 65025$'tir. $2^{15} = 32768 < 65025 < 2^{16} = 65536$ olduğundan 16 bit gerekir.

9.3 4 Basamaklı BCD'den İkiliye Kod Çevirici

Giriş 4 basamaklı bir BCD sayı, çıkış ikili sayıdır. Girişin alabileceği en küçük değer 0, en büyük değer 9999'dur. Çıkış bit sayısı bu en büyük değere göre bulunur.

Çıkış: $2^{13} = 8192 < 9999 < 2^{14} = 16384$ olduğundan 14 bit gerekir (ara adım eklendi). $9999 = 10011100001111_2$.

Giriş: BCD'de her basamak 4 bittir, $4 \times 4 = 16$ bit.

$$\text{ROM boyutu} = 2^{16} \times 14$$

9.4 Özet: ROM Bir Lookup Table'dır

Hocanın vurgusu: ROM, bir lookup table gibi bir tablo yapısıdır. Gerçekleştirmek istediğiniz devrenin doğruluk tablosunu ROM tablosuna yazarsınız. Verdiğiniz adreste hangi değer varsa çıkışta onu görürsünüz.

Devre	Giriş (k)	Çıkış (n)	ROM
3 bitlik sayının karesi (optimize)	3	4	$2^3 \times 4$
8 bitlik toplayıcı/çıkarıcı	18	9	$2^{18} \times 9$
8 bitlik çarpıcı	16	16	$2^{16} \times 16$
4 basamaklı BCD'den ikiliye	16	14	$2^{16} \times 14$

10. PLA (Programmable Logic Array)

10.1 Yapı

PLA'da hem AND kapılarının önünde hem OR kapılarının önünde programlanabilir sigortalar vardır. ROM ve PAL'e göre daha esnek olması bundandır. Hoca yapıyı doğrudan bir örnek üzerinden anlattı.

10.2 Örnek: İki Çıkışlı Fonksiyon

Üç değişkenli (A, B, C) iki fonksiyon indirgenmiş halde verilmiştir:

$$F_1 = AB' + AC + A'BC'$$

$$F_2 = (AC + BC)'$$

Birden fazla çıkış varsa ortak terimler işimizi kolaylaştırır. Burada AC iki fonksiyonda da vardır. Bu, tasarımın daha az malzemeyle oluşmasını sağlar.

Girişler: Üç girişin her biri için inverting buffer vardır. Yani A'nın hem kendisi hem A' üretilir; B ve C için de aynısı geçerlidir. Fonksiyonlarda ikisinden biri gerekebileceği için bu yapı entegrede hazır bulunur.

AND dizisi: Her çarpım terimi (product term) için bir AND kapısı kullanılır. Kapının girişinde gereken değişkenlerin sigortaları kapalı bırakılır, diğerleri açılır.

OR dizisi: Her fonksiyonun OR kapısına hangi çarpım terimlerinin gireceği sigortalarla belirlenir.

Çıkış katmanı: Her çıkış sabit bir XOR kapısından geçer. XOR'un diğer girişi 0 ise OR çıkışı aynen aktarılır, 1 ise tümleyeni alınır. F_2'de OR işleminden sonra tümleyen gerektiği için bu yapı kullanılır.

Örneğin programlanması şöyle özetlenir (hocanın anlatımından tabloya dökülmüştür):

AND kapısı	Çarpım terimi	Kapalı sigortalar	F_1 OR'una bağlı	F_2 OR'una bağlı
1	AB'	A, B'	×	
2	AC	A, C	×	×
3	BC	B, C		×
4	$A'BC'$	A', B, C'	×	

Çıkış	XOR'un diğer girişi	Sonuç
F_1	0	OR çıkışı aynen geçer
F_2	1	OR çıkışının tümleyeni alınır

Dört AND kapısı yeter. Bu programlamanın F_1 ve F_2'yi verdiği sekiz giriş kombinasyonunda python ile doğrulandı.

Giriş bufferları, AND kapıları, OR kapıları ve XOR'lar yapıda hazırdır. Tasarımcı yalnızca modülün kaç AND kapısına ve kaç OR kapısına ihtiyacı olduğunu söyler ve sigortaların konumunu ayarlar. Bunu bir PLA tablosuyla ya da doğrudan yapı üzerinde çarpıları işaretleyerek yapar.

11. PLA Tasarımında Ortak Terim Aramak

> **Hocanın vurgusu:** PLA ile tasarımda önemli kriterlerden biri, fonksiyonlarda olabildiğince fazla ortak terim yakalamaktır. Ortak terim sayısı ne kadar fazlaysa o kadar az AND kapısına ihtiyaç duyarsınız.

Örnekte iki fonksiyon mintermlerle verilmiştir:

$$F_1 = \Sigma(3, 5, 6, 7), \qquad F_2 = \Sigma(1, 2, 3, 7)$$

11.1 Doğrudan Seçim: 5 AND Kapısı

Karnaugh haritalarında ilgili gözler 1 yapılır ve gruplanır:

$$F_1 = AB + AC + BC$$

$$F_2 = A'B + A'C + BC$$

Yalnız BC ortaktır. F_1 ve F_2 çıkışları doğrudan seçilirse toplam 5 AND kapısı gerekir.

11.2 Tümleyeni Seçmek: 4 AND Kapısı

Çıkış katmanındaki XOR ile tümleyen de alınabildiğinden bir alternatifimiz daha vardır. Ortak terim sayısını en fazla yapacak çözümlere bakarız. Örnekte F_2 doğrudan alınır, F_1'in tümleyeni kullanılır ve XOR ile çıkışta F_1 elde edilir.

Burada dikkat edilecek şey, F_2'nin indirgenmiş haliyle değil tablodan "tek tek" alınmış olmasıdır. Hoca önce indirgenmiş ifadelerde ortak terim arayıp bulamadı, sonra bir öğrenci bunu fark etti. Böyle bakınca birinci ve ikinci terimler ortak çıkar. Ortak terim sayısı 2 olur ve fonksiyonlar 4 AND kapısıyla gerçeklenir.

> **Ders sonrası not:** $F_1' = \Sigma(0, 1, 2, 4)$ ile F_2 için en az terimli çözüm tüm çarpım terimleri denenerek bulundu ve 4 terimli tek çözüm şudur:
>
> $$F_1' = A'B'C + A'BC' + B'C', \qquad F_2 = A'B'C + A'BC' + BC$$
>
> Ortak terimler $A'B'C$ ve $A'BC'$'dir, yani 1 ve 2 numaralı mintermlerdir. F_1''in indirgenmiş hali olan $A'B' + A'C' + B'C'$ ile F_2'nin indirgenmiş hali arasında ortak terim yoktur.
>
> **Ders sonrası not:** Aynı taramanın dört polarite için sonucu: (F_1, F_2) için en az 5, (F_1', F_2) için 4, (F_1, F_2') için 4, (F_1', F_2') için 5 AND kapısı.

Soru: Bir de inverter eklememiz gerekmeyecek mi? O zaman yine 5 olmuyor mu? **Cevap:** Hayır. XOR kapıları sistemin içinde mevcuttur. İki çıkışlı bir devre tasarlayacağız diyorsak yapı iki XOR'lu olarak verilir. Değişkenlik arz eden, kaç çarpım terimi yani AND kapısı kullanıldığıdır. Ne kadar az kullanırsak o kadar avantajlıyız.

Bu yüzden fonksiyonları yazarken aralarında ortak terim ararız. Gerekirse indirgenmiş fonksiyondan gitmeyip daha karmaşık halini yazar ve ortak terim olup olmadığına bakarız.

Hoca aynı tarzda bir örneği de deneme için bıraktı: Kaç ortak terim çıkarılabildiğine bakılacak.

Dersin Toparlaması

Dersin ilk bölümünde geçiş tablosu kullanılarak JK flip-flop'la, doğrudan bit değişimine bakılarak da T flip-flop'la sayıcı tasarımı görüldü. T flip-flop örneğinde fark etmez seçiminin kullanılmayan durumlardaki davranışı belirlediği ve $3 \to 5 \to 6 \to 1$ zinciriyle devrenin kendi kendini düzelttiği gösterildi. Tasarımdan sonra bu kontrol yapılmalı, gerekirse fark etmezler değiştirilmelidir.

İkinci bölümde programlanabilir lojik cihazların AND ve OR dizilerinden oluşan ortak yapısı anlatıldı. ROM, sabit bir decoder ve programlanabilir OR dizisiyle çalışan bir lookup table olarak ele alındı. Kare alan devre ve boyut sorularıyla bir doğruluk tablosunun $2^k \times n$ ROM'a nasıl yerleştirildiği görüldü. PLA'da hem AND hem OR dizisi programlanabilir, çıkıştaki XOR ile tümleyen seçilebilir; bu nedenle ortak terim aramak AND kapısı sayısını azaltır. Yetişmeyen PAL yapısı ve küçük bir örneği önümüzdeki haftaya kaldı.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.